SC2006 Software Engineering

Group 2

Lab 4 Coding Agent Exercise

Table of Contents

1. AI Agent tries to recreate RecommendationEngine.java **2**

2. Further Experiment (Feature Implementation) **11**

3. Reflection for Experiment (Section 1) **11**

3.1 Prompting and Materials 11

3.2 Outcome Comparison 12

4. Reflection for Experiment (Section 2) **14**

4.1 Prompting and Materials 14

4.2 Outcome and Expectations 14

1. AI Agent tries to recreate RecommendationEngine.java

Below is the code generated by the AI Agent:

```
// HabitatHero/src/main/java/habitathero/control/RecommendationEngine.java
package habitathero.control;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.stream.Collectors;

import org.json.JSONObject;
import org.springframework.stereotype.Service;

import habitathero.GeoSpatialAnalysis.src.HDBBuildingMgr;
import habitathero.GeoSpatialAnalysis.src.TransportLineMgr;
import habitathero.boundary.RecommendationRequest;
import habitathero.entity.AmenityLocation;
import habitathero.entity.CommuterProfile;
import habitathero.entity.HDBBlock;
import habitathero.entity.HDBDataConstants;
import habitathero.entity.StructuralConstraints;
import habitathero.entity.WeightedPreference;
import habitathero.exception.ZeroMatchesException;
import habitathero.repository.ResaleTransactionRepository;

@Service
public class RecommendationEngine {

    private static final String FACTOR_SOLAR = "Solar Orientation";
    private static final String FACTOR_ACOUSTIC = "Acoustic Comfort";
    private static final String FACTOR_CONVENIENCE = "Convenience";

    private static final double STRICT_NOISE_DISTANCE_METERS = 100.0;
    private static final double STRICT_CONVENIENCE_MIN_SCORE = 0.50;
    private static final double QUIET_NOISE_DB = 55.0;
    private static final double LOUD_NOISE_DB = 85.0;

    private final ResaleTransactionRepository resaleTransactionRepository;
    private final MultiCommuterService multiCommuterService;
    private final HDBBuildingMgr hdbBuildingMgr;
    private final TransportLineMgr transportLineMgr;
    private final ConvenienceScoringService convenienceScoringService;
    private final RankingStrategy rankingStrategy;

    public RecommendationEngine(
            ResaleTransactionRepository resaleTransactionRepository,
            MultiCommuterService multiCommuterService,
            HDBBuildingMgr hdbBuildingMgr,
```

```
            TransportLineMgr transportLineMgr,
            ConvenienceScoringService convenienceScoringService,
            RankingStrategy rankingStrategy) {
        this.resaleTransactionRepository = resaleTransactionRepository;
        this.multiCommuterService = multiCommuterService;
        this.hdbBuildingMgr = hdbBuildingMgr;
        this.transportLineMgr = transportLineMgr;
        this.convenienceScoringService = convenienceScoringService;
        this.rankingStrategy = rankingStrategy;
    }

    public List<HDBBlock> generateRecommendations(RecommendationRequest request) {
        if (request == null) {
            throw new IllegalArgumentException("Request body is missing user profile data.");
        }

        StructuralConstraints constraints = request.getStructuralConstraints();
        if (constraints == null) {
                    throw new IllegalArgumentException("Missing structuralConstraints in request
payload.");
        }

        validateStructuralConstraints(constraints);

        List<BlockCandidateView> candidateList = fetchCandidates(constraints);
        if (candidateList.isEmpty()) {
            throw new ZeroMatchesException();
        }

        Map<String, ResaleTransactionRepository.TownPsfView> townPriceBenchmarks =
                loadTownPriceBenchmarks(candidateList);

        boolean hasValidCommuterPair = hasValidCommuterPair(request.getCommuterProfile());
        if (hasValidCommuterPair) {
                                        multiCommuterService.annotateCommuteScores(candidateList,
request.getCommuterProfile());
        }

        List<WeightedPreference> softConstraints =
                request.getSoftConstraints() == null ? List.of() : request.getSoftConstraints();

        FactorConfig solarConfig = resolveFactorConfig(softConstraints, FACTOR_SOLAR);
        FactorConfig acousticConfig = resolveFactorConfig(softConstraints, FACTOR_ACOUSTIC);
        FactorConfig convenienceConfig = resolveConvenienceConfig(request, softConstraints);

        boolean needsSolarAnalysis = solarConfig.mode != FactorMode.IGNORE;
        boolean needsAcousticAnalysis = acousticConfig.mode != FactorMode.IGNORE;
        int selectedConvenienceCount = countSelectedConvenienceInputs(request);

        List<HDBBlock> rankedBlocks = new ArrayList<>();

        for (BlockCandidateView candidate : candidateList) {
            HDBBlock block = candidate.getBlock();
```

```
            if (block == null) {
                continue;
            }

            JSONObject sunFacingResult = needsSolarAnalysis ? getSunFacingResult(block) : null;
              JSONObject noiseLevelResult = needsAcousticAnalysis ? getNoiseLevelResult(block) :
null;

               double solarScore = needsSolarAnalysis ? scoreSolarOrientation(sunFacingResult) :
0.0;
             double acousticScore = needsAcousticAnalysis ? scoreAcousticComfort(noiseLevelResult)
: 0.0;

            ConvenienceScoringService.ConvenienceEvaluation convenienceEvaluation =
                    convenienceScoringService.evaluateBlock(block, request);
            double convenienceScore = clamp01(convenienceEvaluation.getScore());

            if (!passesStrictConstraints(
                    solarConfig,
                    acousticConfig,
                    convenienceConfig,
                    selectedConvenienceCount,
                    solarScore,
                    acousticScore,
                    convenienceScore,
                    noiseLevelResult)) {
                continue;
            }

            WeightedScore livabilityScore = calculateWeightedLivabilityScore(
                    solarConfig,
                    acousticConfig,
                    convenienceConfig,
                    solarScore,
                    acousticScore,
                    convenienceScore);

            double normalizedLivability = livabilityScore.totalWeight > 0.0
                    ? livabilityScore.weightedScore / livabilityScore.totalWeight
                    : 1.0;
            normalizedLivability = clamp01(normalizedLivability);

            double combinedScore = hasValidCommuterPair
                        ? clamp01((normalizedLivability + clamp01(candidate.getCommuteScore())) /
2.0)
                    : normalizedLivability;

                                enrichBlockFromCandidate(block,  candidate,  convenienceEvaluation,
townPriceBenchmarks);
            block.setGlobalMatchIndex(100.0 * combinedScore);

            rankedBlocks.add(block);
        }
```

```
        if (rankedBlocks.isEmpty()) {
            throw new ZeroMatchesException();
        }

        return rankingStrategy.rank(rankedBlocks, request);
    }

    private void validateStructuralConstraints(StructuralConstraints constraints) {
        String flatType = constraints.getPreferredFlatType();
                                if    (flatType    !=    null    &&    !flatType.isBlank()    &&
!HDBDataConstants.isValidFlatType(flatType)) {
            throw new IllegalArgumentException("Invalid flat type: " + flatType);
        }

        List<String> towns = constraints.getPreferredTowns();
        if (towns == null) {
            return;
        }

        for (String town : towns) {
            if (town == null || town.isBlank()) {
                continue;
            }
            if (!HDBDataConstants.isValidTown(town.trim())) {
                throw new IllegalArgumentException("Invalid town: " + town);
            }
        }
    }

    private List<BlockCandidateView> fetchCandidates(StructuralConstraints constraints) {
             double  maxBudget  =  constraints.getMaxBudget()  >  0  ?  constraints.getMaxBudget()  :
Double.MAX_VALUE;
        int minLeaseYears = Math.max(0, constraints.getMinLeaseYears());
        String preferredFlatType = constraints.getPreferredFlatType();

        boolean townFilterDisabled =
                                                constraints.getPreferredTowns()   ==   null   ||
constraints.getPreferredTowns().isEmpty();

        List<String> preferredTowns = townFilterDisabled
                ? List.of("__TOWN_FILTER_DISABLED__")
                : constraints.getPreferredTowns().stream()
                        .filter(town -> town != null && !town.isBlank())
                        .map(town -> town.trim().toUpperCase(Locale.ROOT))
                        .collect(Collectors.toList());

        return resaleTransactionRepository.findCandidateBlocks(
                maxBudget,
                minLeaseYears,
                preferredFlatType,
                preferredTowns,
                townFilterDisabled);
```

```
    }

    private Map<String, ResaleTransactionRepository.TownPsfView> loadTownPriceBenchmarks(
            List<BlockCandidateView> candidateList) {
        List<String> towns = candidateList.stream()
                .map(BlockCandidateView::getBlock)
                              .filter(block  ->  block  !=  null  &&  block.getTown()  !=  null  &&
!block.getTown().isBlank())
                .map(HDBBlock::getTown)
                .map(this::normalizeTownKey)
                .distinct()
                .collect(Collectors.toList());

        if (towns.isEmpty()) {
            return Map.of();
        }

        return resaleTransactionRepository.findAveragePsfByTowns(towns).stream()
                .filter(view -> view.getTown() != null && view.getAvgPsf() != null)
                .collect(Collectors.toMap(
                        view -> normalizeTownKey(view.getTown()),
                        view -> view,
                        (left, right) -> left,
                        LinkedHashMap::new));
    }

    private boolean hasValidCommuterPair(CommuterProfile commuterProfile) {
        return commuterProfile != null
                && commuterProfile.isEnabled()
                && commuterProfile.getDestinationA() != null
                && commuterProfile.getDestinationB() != null;
    }

      private  FactorConfig  resolveFactorConfig(List<WeightedPreference>  softConstraints,  String
factorName) {
        Optional<WeightedPreference> match = softConstraints.stream()
                .filter(pref -> pref != null && pref.getFactorName() != null)
                .filter(pref -> factorName.equalsIgnoreCase(pref.getFactorName().trim()))
                .findFirst();

        if (match.isEmpty()) {
            return new FactorConfig(FactorMode.IGNORE, 0.0);
        }

        WeightedPreference pref = match.get();
        if (pref.isStrict()) {
            return new FactorConfig(FactorMode.STRICT, 1.0);
        }

        double weight = Math.max(0.0, pref.getPriorityWeight());
        if (weight <= 0.0) {
            return new FactorConfig(FactorMode.IGNORE, 0.0);
        }
```

```
        return new FactorConfig(FactorMode.WEIGHTED, weight);
    }

    private FactorConfig resolveConvenienceConfig(
            RecommendationRequest request,
            List<WeightedPreference> softConstraints) {
        String modeRaw = request.getConvenienceMode();
        if (modeRaw != null && !modeRaw.isBlank()) {
            String mode = modeRaw.trim().toLowerCase(Locale.ROOT);
            if (mode.contains("ignore")) {
                return new FactorConfig(FactorMode.IGNORE, 0.0);
            }
            if (mode.contains("strict")) {
                return new FactorConfig(FactorMode.STRICT, 1.0);
            }
            if (mode.contains("weight")) {
                                return  new  FactorConfig(FactorMode.WEIGHTED,  Math.max(0.0,
request.getConvenienceWeight()));
            }
        }

        return resolveFactorConfig(softConstraints, FACTOR_CONVENIENCE);
    }

    private int countSelectedConvenienceInputs(RecommendationRequest request) {
        int amenityCount = request.getSelectedAmenities() == null
                ? 0
                : (int) request.getSelectedAmenities().stream()
                        .filter(item -> item != null && !item.trim().isEmpty())
                        .count();

                         int   parentsCount   =   request.getParentsPostalCode()   !=   null   &&
!request.getParentsPostalCode().isBlank() ? 1 : 0;
        return amenityCount + parentsCount;
    }

    private JSONObject getSunFacingResult(HDBBlock block) {
        if (block.getPostalCode() == null || block.getPostalCode().isBlank()) {
            return null;
        }
        return hdbBuildingMgr.getSunFacing(block.getPostalCode());
    }

    private JSONObject getNoiseLevelResult(HDBBlock block) {
        if (block.getPostalCode() == null || block.getPostalCode().isBlank()) {
            return null;
        }
        return transportLineMgr.getNoiseLevel(block.getPostalCode());
    }

    private double scoreSolarOrientation(JSONObject sunFacingResult) {
```

```
        if (sunFacingResult == null || !"OK".equalsIgnoreCase(sunFacingResult.optString("status",
"))) {
            return 0.0;
        }

        double westExposurePct =
                firstFinite(
                        sunFacingResult.optDouble("westScoreRelativeExposurePct", Double.NaN),
                        sunFacingResult.optDouble("westRatio", Double.NaN) * 100.0,
                        Double.NaN);

        if (!Double.isFinite(westExposurePct)) {
            return 0.0;
        }

        return clamp01(1.0 - (westExposurePct / 100.0));
    }

    private double scoreAcousticComfort(JSONObject noiseLevelResult) {
                                                if      (noiseLevelResult      ==      null      ||
!"OK".equalsIgnoreCase(noiseLevelResult.optString("status", "))) {
            return 0.0;
        }

        double noiseDb = noiseLevelResult.optDouble("noise_level_db", Double.NaN);
        if (!Double.isFinite(noiseDb)) {
            return 0.0;
        }

        if (noiseDb <= QUIET_NOISE_DB) {
            return 1.0;
        }
        if (noiseDb >= LOUD_NOISE_DB) {
            return 0.0;
        }

        return clamp01(1.0 - ((noiseDb - QUIET_NOISE_DB) / (LOUD_NOISE_DB - QUIET_NOISE_DB)));
    }

    private boolean passesStrictConstraints(
            FactorConfig solarConfig,
            FactorConfig acousticConfig,
            FactorConfig convenienceConfig,
            int selectedConvenienceCount,
            double solarScore,
            double acousticScore,
            double convenienceScore,
            JSONObject noiseLevelResult) {

        if (solarConfig.mode == FactorMode.STRICT && solarScore < 0.50) {
            return false;
        }
```

```
        if (acousticConfig.mode == FactorMode.STRICT) {
            double distanceMeters = noiseLevelResult == null
                    ? Double.NaN
                    : noiseLevelResult.optDouble("distance_meters", Double.NaN);
                                if   (!Double.isFinite(distanceMeters)   ||   distanceMeters   <
STRICT_NOISE_DISTANCE_METERS) {
                return false;
            }
        }

        if (convenienceConfig.mode == FactorMode.STRICT
                && selectedConvenienceCount > 0
                && convenienceScore < STRICT_CONVENIENCE_MIN_SCORE) {
            return false;
        }

        return true;
    }

    private WeightedScore calculateWeightedLivabilityScore(
            FactorConfig solarConfig,
            FactorConfig acousticConfig,
            FactorConfig convenienceConfig,
            double solarScore,
            double acousticScore,
            double convenienceScore) {

        double weightedScore = 0.0;
        double totalWeight = 0.0;

        if (solarConfig.mode == FactorMode.WEIGHTED) {
            weightedScore += solarScore * solarConfig.weight;
            totalWeight += solarConfig.weight;
        }

        if (acousticConfig.mode == FactorMode.WEIGHTED) {
            weightedScore += acousticScore * acousticConfig.weight;
            totalWeight += acousticConfig.weight;
        }

        if (convenienceConfig.mode == FactorMode.WEIGHTED) {
            weightedScore += convenienceScore * convenienceConfig.weight;
            totalWeight += convenienceConfig.weight;
        }

        return new WeightedScore(weightedScore, totalWeight);
    }

    private void enrichBlockFromCandidate(
            HDBBlock block,
            BlockCandidateView candidate,
            ConvenienceScoringService.ConvenienceEvaluation convenienceEvaluation,
            Map<String, ResaleTransactionRepository.TownPsfView> townPriceBenchmarks) {
```

```
        block.setEstimatedPrice(candidate.getAverageResalePrice());
        block.setFloorAreaSqm(candidate.getAverageFloorAreaSqm());
        block.setRemainingLeaseYears((int) Math.round(candidate.getAverageRemainingLease()));
        block.setConvenienceScore(clamp01(convenienceEvaluation.getScore()));
        block.setConvenienceFactors(convenienceEvaluation.getAmenityMatches());
        block.setMatchedAmenities(convenienceEvaluation.getMatchedAmenities());

        String townKey = normalizeTownKey(block.getTown());
        ResaleTransactionRepository.TownPsfView townBenchmark = townPriceBenchmarks.get(townKey);
        if (townBenchmark != null && townBenchmark.getAvgPsf() != null) {
            block.setTownAveragePsf(townBenchmark.getAvgPsf());
            block.setTownTransactionCount(
                                    townBenchmark.getTransactionCount()  ==  null  ?  0L  :
townBenchmark.getTransactionCount());
        }

        Map<String, AmenityLocation> nearestAmenities = new LinkedHashMap<>();
        if (convenienceEvaluation.getNearestAmenities() != null) {
            nearestAmenities.putAll(convenienceEvaluation.getNearestAmenities());
        }

        if (!nearestAmenities.isEmpty()) {
            block.setNearestAmenities(nearestAmenities);
        }
    }

    private String normalizeTownKey(String value) {
        if (value == null) {
            return null;
        }
        String trimmed = value.trim();
        return trimmed.isEmpty() ? null : trimmed.toUpperCase(Locale.ROOT);
    }

    private double clamp01(double value) {
        if (!Double.isFinite(value)) {
            return 0.0;
        }
        return Math.max(0.0, Math.min(1.0, value));
    }

    private double firstFinite(double... values) {
        for (double value : values) {
            if (Double.isFinite(value)) {
                return value;
            }
        }
        return Double.NaN;
    }

    private enum FactorMode {
        IGNORE,
```

```
        STRICT,
        WEIGHTED
    }

    private record FactorConfig(FactorMode mode, double weight) {}

    private record WeightedScore(double weightedScore, double totalWeight) {}
}
```

2. Further Experiment (Feature Implementation)

For our further experiment, we asked the AI assistant to implement account security hardening by adding two protections: a password change function for authenticated users and a login lockout mechanism after repeated failed login attempts. We provided the assistant with our existing backend and frontend codebase context and asked it to integrate the feature into our current authentication flow instead of creating a separate module.

We prompted the assistant to:

1. Add a secure change-password endpoint that verifies the old password before allowing a new password update.
2. Add failed login tracking and temporary account lockout after multiple incorrect password attempts.
3. Update the profile page UI so users can submit old and new password values.
4. Keep the behavior consistent with our current JWT-based authentication setup.

The implemented outcome included:

1. A change-password API route in the authentication controller.
2. Failed-attempt counting and temporary lockout logic in login.
3. Frontend support in profile settings for password update submission.
4. User-facing lockout/error messages when login limits are exceeded.
5. This experiment is suitable as a “new feature not included in earlier scope detail” because it goes beyond basic registration/login by introducing defensive authentication controls.

3. Reflection for Experiment (Section 1)

3.1 Prompting and Materials

Prompting and Materials: We utilized the Cline agent within VS Code for this exercise. After temporarily deleting the `RecommendationEngine.java` file, we provided Cline with the following context: the `RecommendationRequest` boundary, the `HDBBlock` entity, the `ResaleTransactionRepository` interface, and our class diagrams. We prompted:

> *"I have deleted our core control class, `RecommendationEngine.java`. Using the provided repository and boundary files as context, please recreate the class from scratch. It must handle structural constraint filtering, weighted livability scoring for*

solar and noise, and return a ranked list of HDB blocks." We performed one iteration where we had to remind the agent to use our existing `TransportLineMgr` for the noise-level logic rather than inventing a new service.

3.2 Outcome Comparison

They are very similar at the top level, but the generated version is a bit more generic, while our real RecommendationEngine contains several project-specific fallback rules and stricter domain behavior.

The same parts are:

constructor dependencies
main generateRecommendations() flow
request / structural constraint validation
candidate retrieval
optional two-commuter scoring
factor config resolution for Solar, Acoustic, and Convenience
weighted livability computation
final GlobalMatchIndex calculation
town benchmark enrichment and nearest-amenity enrichment

The main differences are these:

Our real code is more defensive with fallback behavior.
In the actual file, getSunFacingResult() and getNoiseLevelResult() normalize the postal code first and catch exceptions, returning null on failure. The simulated version also catches errors, but it does not match the exact helper flow used in our real file.

Our real code has domain-specific fallback scoring for both solar and acoustic comfort.
For solar, our actual code first checks geospatial output, then falls back to the persisted block.isWestSunStatus() field if geospatial output is unavailable. It also handles "WEST", "EAST", and "BALANCED" explicitly. The simulated version was simpler and would miss some of that exact behavior.

For acoustic comfort, our real code is noticeably more specific.
It uses a separate scoreFromNoiseDb() helper, checks !noiseLevelResult.has("error"), and if live geospatial output is missing, it falls back to the persisted block field noiseRiskLevel == "Low". The simulated code treated missing geospatial output more harshly and did not mirror that exact fallback logic.

Our real strict filtering is also more accurate.

In the actual file, passesStrictConstraints() does not directly compare solarScore and acousticScore alone. Instead, it delegates to passesStrictSolar() and passesStrictAcoustic(). That matters because:

strict solar checks dominant first, then falls back to score logic, then finally falls back to westSunStatus
strict acoustic first checks distance_meters >= 100, then falls back to noise-score threshold, then finally falls back to persisted "Low" noise risk

The simulated code simplified this and would miss some valid passes or fails compared to our real implementation.

Our real convenience handling is slightly different too.
In the actual file, resolveConvenienceConfig() supports an explicit convenience mode from the request by calling normalizeMode(request.getConvenienceMode()), converting that to FactorMode, and only using convenienceWeight when the mode is WEIGHTED. The simulated code used looser string matching like "ignore", "strict", and "weight", which is plausible, but not the same as our actual implementation.

The simulated code also slightly over-counted convenience inputs.
Our actual code counts selectedAmenities only when computing selectedConvenienceCount. The simulated version also counted parentsPostalCode, which would make strict convenience filtering harsher than our real code.

Our real code also has a specific MRT enrichment detail.
When noiseLevelResult contains distance_meters, the actual engine adds a synthetic "mrtStation" entry into nearestAmenities using that transport distance. That detail was missing or less exact in the simulated reconstruction.

There is also a small weighting difference.
In our actual resolveFactorConfig(), a strict factor gets new FactorConfig(FactorMode.STRICT, 0.0), while the simulated version used weight 1.0 for strict in some places. That usually would not break the code because strict factors are filtered separately, but it is still not identical to our real implementation.

4. Reflection for Experiment (Section 2)

Chosen Experiment: Feature Implementation (Account Security Hardening).

4.1 Prompting and Materials

We tasked the AI with implementing defensive authentication controls. We provided the `AuthController.java`, `User.java` entity, and our `SecurityConfig.java` to ensure it understood our JWT filter chain. We prompted:

> *"Implement two security features: 1) A `/change-password` endpoint that validates the current password before updating, and 2) A lockout mechanism that temporarily disables an account after 5 failed login attempts. Update both the Backend controller and provide a React snippet for the Profile settings page."*

4.2 Outcome and Expectations

The outcome of this experiment was a functional integration of account security hardening features that aligned with our existing architecture. The AI assistant correctly proposed adding a `failed_attempts` column and a `lockout_until` timestamp to our **User** entity. It provided the necessary logic within the authentication flow to increment these attempts upon a failed login and verify the lockout status before permitting a JWT to be issued.

A significant technical success was the assistant's ability to utilize `BCryptPasswordEncoder` for the password-change logic without being explicitly told, ensuring that our security implementation remained compliant with the non-functional requirements (NFRs) defined in our SRS. The generated React snippet for the **Profile** page also accurately mirrored our existing state management patterns, allowing for a quick "drop-in" implementation of the new form.

Our expectations were largely met, particularly regarding implementation speed. Generating the boilerplate for password verification, failed-state tracking, and frontend form wiring would have taken significantly longer manually. However, human oversight remained essential in two key areas:

1. **Security Policy Refinement:** We had to manually adjust the error messages to be generic (e.g., "Invalid credentials") to prevent "Username Enumeration" vulnerabilities, as the AI initially suggested specific messages like "User not found."
2. **Persistence Logic:** We required an additional iteration to ensure that the lockout duration was handled correctly on the backend to avoid edge cases where a server restart might inadvertently clear active lockouts stored only in volatile memory.

Ultimately, the experiment confirmed that while AI assistance is an exceptional tool for rapid feature scaffolding and "speed-to-code," developer expertise is mandatory for final security audits and ensuring alignment with professional UX standards